managed here, and it just toggles state from unsold to held and vice versa. The actual click handler onSeatChange is a little more complicated.

The first thing onSeatChange does is if you click an invalid seat, the function just returns without doing anything. If we click on a valid seat, we then need to change the seat statuses in such a way that React will trigger re-render.

Our partner here is setSeatStatuses, the function created by the useState hook at the beginning of the component. In order to call this function in such a way as to trigger React, we cannot use the existing seatStatuses array with values changed; React will not recognize that as a new value.

Instead, we have to build a completely new array and pass the new array to setSeatStatuses. We do this by using the map function to create a new array from the old array. For each seat, if the seat number is either the seat that was clicked or a seat to its right based on the number of tickets being bought, we toggle its state. Otherwise, we just pass the state through. The result of this map is that we wind up with a new array with new values for the affected seats, but the same value for the unaffected seats.

Re-rendering the row causes all the Seat components to be redrawn. Now that Seat no longer manages its own state, the component is somewhat simpler:

chapter_04/04/app/javascript/components/seat.tsx

```
import * as React from "react"

interface SeatProps {
  seatNumber: number
  status: string
  clickHandler: (seatNumber: number) => void
}

export const Seat = ({
  seatNumber,
  status,
  clickHandler,
}: SeatProps): React.ReactElement => {
  function stateDisplayClass(): string {
```